

Student Performance Analyzer in Java

Java Code

```
import java.util.*;
import java.util.stream.*;

class Student {
    private int id;
    private String name;
    private List<String> courses;
    private Map<String, Integer> scores;

    public Student(int id, String name, List<String> courses,
        Map<String, Integer> scores) {
        this.id = id;
        this.name = name;
        this.courses = courses;
        this.scores = scores;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public List<String> getCourses() {
        return courses;
    }

    public Map<String, Integer> getScores() {
        return scores;
    }

    public double getAverageScore() {
        return scores.values()
            .stream()
            .mapToInt(Integer::intValue)
            .average()
            .orElse(0.0);
    }
}

public class StudentPerformanceAnalyzer {

    public static List<Student> getTopNStudents(
        List<Student> students, int n) {

        return students.stream()
            .sorted(Comparator.comparingDouble(
                Student::getAverageScore).reversed())
            .limit(n)
            .collect(Collectors.toList());
    }

    public static Map<String, Double> getAverageScorePerCourse(
        List<Student> students) {

        Map<String, Double> result = new HashMap<>();

        Set<String> courses = getAllUniqueCourses(students);

        for (String course : courses) {
            double avg = students.stream()
                .mapToInt(s -> s.getScores()
                    .getOrDefault(course, 0))
                .average()
                .orElse(0.0);

            result.put(course, avg);
        }
    }
}
```

```

    }
    return result;
}

public static Set<String> getAllUniqueCourses(
    List<Student> students) {

    return students.stream()
        .flatMap(s -> s.getCourses().stream())
        .collect(Collectors.toCollection(HashSet::new));
}

public static void main(String[] args) {

    List<Student> students = new ArrayList<>();

    Map<String, Integer> scores1 = new HashMap<>();
    scores1.put("Java", 85);
    scores1.put("DBMS", 90);

    Map<String, Integer> scores2 = new HashMap<>();
    scores2.put("Java", 95);
    scores2.put("OS", 88);

    students.add(new Student(
        1,
        "Alice",
        Arrays.asList("Java", "DBMS"),
        scores1));

    students.add(new Student(
        2,
        "Bob",
        Arrays.asList("Java", "OS"),
        scores2));

    System.out.println("Top Students:");
    getTopNStudents(students, 2)
        .forEach(s -> System.out.println(
            s.getName() + " : " +
            s.getAverageScore()));

    System.out.println("\nCourse Averages:");
    System.out.println(
        getAverageScorePerCourse(students));

    System.out.println("\nUnique Courses:");
    System.out.println(
        getAllUniqueCourses(students));
}
}

```

Complexity Analysis

1. Time Complexity of Computing Course Averages:

Let S be the number of students and C be the number of unique courses. For each course, all students are traversed to compute averages. Therefore, the complexity is $O(S \times C)$.

2. Complexity of Sorting Top N Students:

Students are sorted using Comparator based on average score. Sorting S students takes $O(S \log S)$. Selecting the top N students after sorting takes $O(N)$. Overall complexity: $O(S \log S)$.